

Table Of Contents

1.0 Introduction.....	2
2.0 Methodology.....	2
2.1 Research Design and Workflow.....	2
2.2 Experimental Setup.....	3
2.2.1 Hardware Configuration.....	3
2.2.2 Software Environment.....	3
2.3 Dataset Description and Preprocessing.....	3
2.3.1 Data Sources.....	3
2.3.2 Data Preprocessing.....	3
2.3.3 Image Transformation.....	4
2.3.4 Image Normalization.....	4
2.3.5 Data splitting for model training and testing.....	4
2.4 Model Training.....	4
2.4.1 Multiclass SVM.....	5
2.4.1.1 Training the SVM model.....	5
2.4.2 Standard Convolutional Neural Network (CNN).....	6
2.4.2.1 Proposed CNN Architecture.....	6
2.4.2.2 Visual Representation of the CNN Network.....	7
2.4.3. ResNet-50 CNN.....	7
2.4.3.1 Visual Representation of a ResNet-50 Network.....	7
2.4.3.2 Training the network for CNN and ResNet50.....	7
2.4.3.3 Training Hyperparameters.....	7
2.4.4 Hyperparameter Tuning.....	8
2.5 Model Performance Evaluation.....	8
3.0 Results.....	8
3.1 Accuracy.....	8
3.2 The Classification Report.....	8
3.2.1 SVM Classification Report.....	9
3.2.2 The Standard CNN Classification Report.....	9
3.2.3 The ResNet-50 CNN Classification Report.....	9
3.3 The Confusion Matrix.....	10
3.3.1 SVM Confusion Matrix.....	10
3.3.2 Standard CNN Confusion Matrix.....	10
3.3.3 ResNet50 Confusion Matrix.....	11
3.4 Summary.....	11
4.0 Conclusion.....	12
5.0 Future Work.....	12

1.0 Introduction

Agriculture is the backbone of Uganda's economy employing over 70% of the country's population, contributing nearly 25% to the country's GDP and significantly contributing to household food security and income. Among the main crops grown are cassava and tomatoes specifically as staple and cash crops respectively. Cassava provides daily food for millions throughout the year due to its resilience even in critical climatic conditions while tomatoes provide incomes to families as they are a key horticultural crop exported and mainly used in making sauces and soups.

Production of these crops is however threatened by different diseases. Cassava Mosaic Disease and Cassava Brown Streak Disease reduce yields of cassava by over 70% in different regions of the country. Early blight, Late blight, and bacterial wilt devastate tomato production leading to increased financial losses to small scale farmers. These diseases are usually detected late due to reliance on manual inspection, lack of diagnostic tools and limited access to skilled personnel.

Early detection of such diseases can significantly help reduce crop losses, improve yields thus increase incomes and farmer livelihoods. With increased penetration of technology, internet and smartphones, a machine learning disease detection tool presents a timely solution to curbing losses due to late disease detection.

This project aimed to develop a machine learning model to provide timely detection of some of the diseases affecting cassava and tomatoes. The diseases the model was trained to predict are Cassava Mosaic Disease, and Cassava Brown Streak Disease for cassava, and Tomato Early Blight Disease and Tomato Bacterial Spot for tomatoes. It evaluated three models based on three supervised learning algorithms; Multiclass Support Vector Machine (SVM), a Standard Convolutional Neural Network (CNN), and ResNet50; a classical Convolutional Neural Network comparing their performances to determine the most effective model for disease detection.

2.0 Methodology

This section describes the methodological approach used to develop, train, and evaluate machine learning models for multiclass image-based disease detection in cassava and tomato crops. The methodology covers the overall workflow, experimental setup, dataset preparation, model development, training procedures, and evaluation strategy.

2.1 Research Design and Workflow

The project followed a supervised learning workflow designed for image-based classification tasks that consisted of the following stages: Data Acquisition, Data Preprocessing and feature engineering, Model Development, Model Training and Optimization, Model Evaluation and Comparison using quantitative metric, and finally interpretation and reporting. This structured workflow ensured consistency across models and enabled fair comparison of their predictive performance.

2.2 Experimental Setup

2.2.1 Hardware Configuration

All experiments were conducted on a personal computer with the following specifications:

- **Processor:** 2.5 GHz Dual-Core Intel CPU
- **Memory:** 8 GB RAM
- **Storage:** 300 GB SSD
- **GPU:** None (CPU-only execution)

Limited computational resources influenced the methodological decisions, including dataset reduction, simplified hyperparameter tuning, and use of transfer learning.

2.2.2 Software Environment

The software setup consisted of:

- **Operating System:** macOS
- **Programming Language:** Python
- **Notebook Environment:** Jupyter Notebook
- **Libraries:**
 - PyTorch 1.13.1
 - torchvision 0.14.1
 - scikit-learn
 - pandas, numpy, matplotlib
 - pillow for image loading

2.3 Dataset Description and Preprocessing

2.3.1 Data Sources

The project uses datasets from Makerere Artificial Intelligence Lab and NaCRRRI under the Lacuna Project for cassava disease classification and datasets from Plant Village for tomato disease classification. The cassava dataset is a 9116-image dataset hosted at Harvard Dataverse captured in Ugandan farmer fields. It includes 1119 healthy uninfected cassava leaves, 5000 leaves infected by Cassava mosaic disease and 2997 leaves infected by Cassava brown streak disease. The tomato dataset is a multi-image dataset hosted on Kaggle. It includes 1591 healthy uninfected tomato leaves, 2127 leaves infected by Tomato Bacterial Spot, and 1000 leaves infected by Tomato early Blight.

2.3.2 Data Preprocessing

The dataset was broken down into 6 distinct folders for storage of the image files. Due to the large dataset and minimal computational resources, the number of instances in every folder was reduced to about 300 images that could be loaded and executed with the limited available resources. This was

done by removing duplicates and then randomly deleting any other files so as to get to the desired range. This ensured the dataset was balanced and could easily be used in model training despite the possibility of affecting generalization of the model. No image augmentation techniques were applied due to limited computational resources. The folders were then labelled as shown in the table below:

Folder Name	Purpose
Tomato_Healthy	Store all images of healthy tomato leaves
Cassava_Healthy	Store all images of healthy cassava leaves
Tomato_Bacterial_Spot	Store all images of tomato leaves infected with bacterial spot
Tomato_Early_Blight	Store all images of tomato leaves infected with early blight
Cassava_Brown_Streak	Store all images of cassava leaves infected with brown streak disease
Cassava_Mosaic	Store all images of cassava leaves infected with cassava mosaic disease

2.3.3 Image Transformation

The images were resized to 256*256 image dimensions and then transformed into vectors (numpy arrays) storing pixelated data from the images for easy model training for the Support Vector Machine. On the other hand, a copy of the images was resized to 256*256 image dimensions and then transformed into tensors using pytorch for easy training of CNN and ResNet50 models.

2.3.4 Image Normalization

The image data was then normalized to ensure model convergence and prevent overfitting. The vector data was normalized using Standard Scaler. The tensors were also normalized using the Normalize function available in pytorch. This ensured all data was scaled to a range having a mean of 0 and Standard Deviation of -1 to 1.

2.3.5 Data splitting for model training and testing

The data was split into train and test sets using an 80-20 ratio split i.e 80% being used for model training and 20% for model testing and evaluation. The SVM model used sklearn library to perform data split, while the CNN models used the splitfolders library to perform data splitting.

2.4 Model Training

The project trained machine learning models based on three algorithms: Multiclass Support Vector Machine, Standard Convolution Neural Network and ResNet50, a classical CNN algorithm.

2.4.1 Multiclass SVM

Support Vector Machine is a supervised machine learning algorithm used for classification and regression tasks. It works by finding the best hyperplane, or decision boundary, that separates data into different classes while maximizing the distance, or margin, between the hyperplane and the closest data points of each class. SVM is mainly used for binary classification and uses the kernel trick to transform data into a higher dimensional space where a hyperplane can be found making it good for data that is both linear and non-linear.

Because the project was a multiclass classification problem, a different strategy using SVM was devised. SVM employs two primary approaches when dealing with multiclass problems:

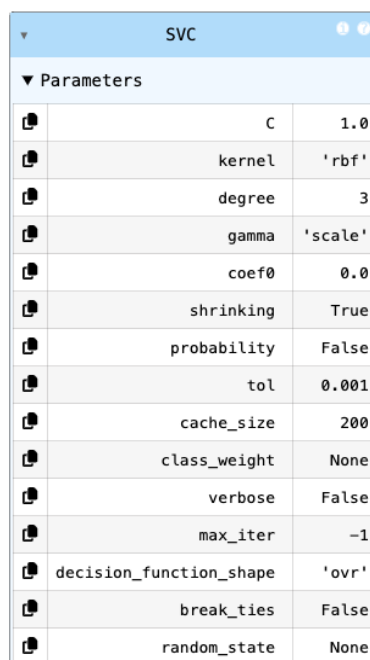
i. One Vs One Approach (OVO)

This works by creating a binary classifier for every possible pair of classes. It is mainly used when the number of classes is relatively small, as it allows for a focused comparison between pairs of classes. Each classifier is responsible for distinguishing between two specific classes, effectively ignoring the others.

ii. One Vs All (OVA)/ One Vs Rest (OVR) Approach

This works by training a single binary classifier for each class where each class is treated as the positive class, while all other classes are grouped together as the negative class.

The project employed the One Vs All (OVA) approach with SVM as the decision function for model development using the svm function available via the sklearn library.

A screenshot of a software interface showing the parameters for an SVC (Support Vector Classification) model. The window title is 'SVC'. Below the title bar is a section labeled 'Parameters' with a downward arrow. A table lists 17 parameters and their values. Each row has a small icon on the left. The parameters are: C (1.0), kernel ('rbf'), degree (3), gamma ('scale'), coef0 (0.0), shrinking (True), probability (False), tol (0.001), cache_size (200), class_weight (None), verbose (False), max_iter (-1), decision_function_shape ('ovr'), break_ties (False), and random_state (None).

SVC		
Parameters		
C		1.0
kernel		'rbf'
degree		3
gamma		'scale'
coef0		0.0
shrinking		True
probability		False
tol		0.001
cache_size		200
class_weight		None
verbose		False
max_iter		-1
decision_function_shape		'ovr'
break_ties		False
random_state		None

2.4.1.1 Training the SVM model

The model was trained on the data using the following set of hyperparameters:

- **C:** The model used 1.0 as the regularization parameter to control the trade-off between achieving the most optimal hyperplane and minimizing training errors.
- **kernel:** The model used the radial basis function (rbf) as a decision boundary to determine the optimal hyperplane between the data samples
- **gamma:** This is the kernel coefficient and determines the influence of a single support vector while determining the hyperplane.

2.4.2 Standard Convolutional Neural Network (CNN)

Convolutional Neural Networks is a type of deep learning model mainly used for processing grid-like data such as images. It uses layers to identify patterns from edges, textures and more complex features. Main components of a CNN are convolution layers, pooling layers, and fully connected layers (dense) layers. These were created using functions available via the pytorch library.

2.4.2.1 Proposed CNN Architecture

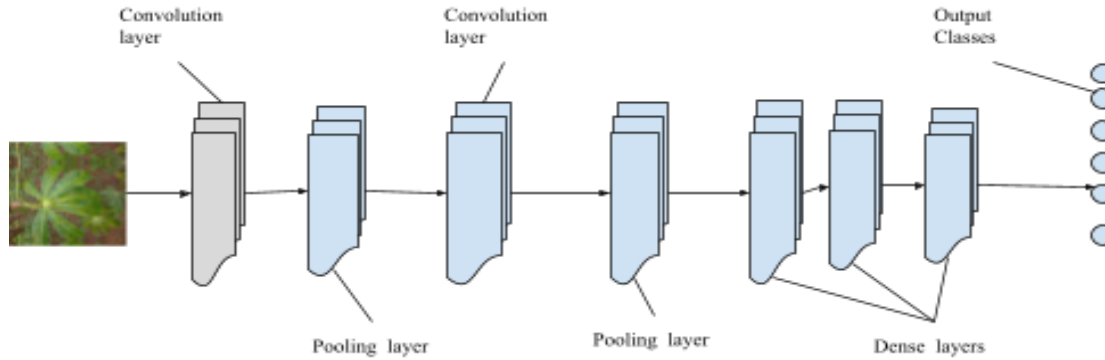
The project used a standard CNN which was layered as follows:

```
CNN(
    (conv1): Conv2d(3, 16, kernel_size=(4, 4), stride=(1, 1), padding=(1, 1))
    (pool): MaxPool2d(kernel_size=4, stride=4, padding=0, dilation=1, ceil_mode=False)
    (conv2): Conv2d(16, 32, kernel_size=(4, 4), stride=(1, 1), padding=(1, 1))
    (fc1): Linear(in_features=7200, out_features=2400, bias=True)
    (fc2): Linear(in_features=2400, out_features=120, bias=True)
    (fc3): Linear(in_features=120, out_features=6, bias=True)
)
```

- **Convolution Layer:** This layer applies filters (kernels) that slide across the input image to detect features like edges, corners and textures. It takes an input image with 3 input channels (RGB), applies a 4*4 filter, and returns a feature map of output data with 16 channels. It uses the Rectified Linear Unit as an activation function.
- **Pooling Layer:** This layer downsamples spatial dimensions (height and width) of the input feature map from the convolution layer by selecting the maximum value within a 4*4 sliding window (kernel).
- **Convolution Layer:** This layer applies a 4*4 filter that slides across the feature map from the pooling layer to detect more advanced features and returns an output with 32 channels. It uses the Rectified Linear Unit as an activation function.
- **Pooling Layer:** This layer again downsamples dimensions of the input feature map from the convolution layer by selecting the maximum value within a 4*4 sliding window (kernel).
- **Fully connected Layers (Dense Layers):** The neural network uses 3 dense layers to perform classification. The first layer receives input from the pooling layer and transforms them into 2400 outfeatures. The second layer receives the 2400 outfeatures and transforms them into 120 outfeatures. The third layer receives the 120 outfeatures and transforms them to produce

6 outfeatures. The model applies the softmax activation function to output data in 6 distinct classes.

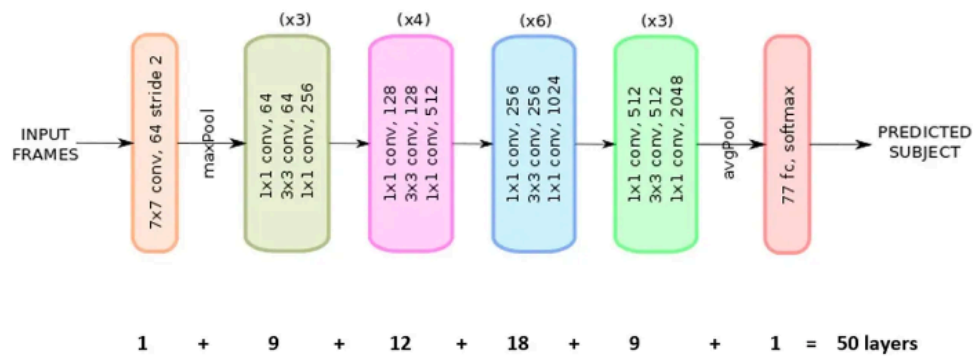
2.4.2.2 Visual Representation of the CNN Network



2.4.3. ResNet-50 CNN

ResNet-50 is a deep Convolutional neural network architecture, with 50 layers deep developed by Microsoft Research for image recognition tasks. It uses residual connections also known as skip connections, which allow it to train more effectively by mitigating the vanishing gradient problem in deep networks. It benefits from transfer learning to perform model training. The project used a pretrained ResNet-50 model from the torchvision library to detect patterns in the dataset.

2.4.3.1 Visual Representation of a ResNet-50 Network



2.4.3.2 Training the network for CNN and ResNet50

1. Forward Pass: Image goes through the CNN and a prediction is made.
2. Loss is calculated using Cross-Entropy Loss.
3. Backpropagation: The gradient for every parameter is fed back into the network.
4. Adam is used as an optimizer to modify all the filters/weights in a direction that reduces the loss.

2.4.3.3 Training Hyperparameters

- **Batch size:** 16 or 32 (depending on memory availability)
- **Epochs:** Both models were trained on 10 epochs.
- **Optimizer:** Adam was used as the primary optimizer for both models.
- **Loss Function:** CrossEntropyLoss
- **Learning Rate:** 0.001 (default), adjusted minimally due to constraints

2.4.4 Hyperparameter Tuning

The project intended to use Grid Search as a hyperparameter tuning method for determining the best hyperparameters that would result in improved performance for both models. However, due to limited computational resources, training the models proved impossible as the notebook would repeatedly close.

2.5 Model Performance Evaluation

The performance of the three models developed was evaluated based on three distinct metrics: Accuracy, the Classification Report and the Confusion Matrix.

3.0 Results

3.1 Accuracy

Accuracy is the degree to which the model predicts true positive classes for every prediction. The accuracy of the three models was calculated and is shown in the table below:

Model	Accuracy (%)
Multiclass Support Vector Machine	75.45219638242894
Standard Convolution Neural Network	73.96907216494846
ResNet-50	81.95876288659794

The accuracy of the models indicated that the ResNet-50 CNN model performed better with over 81% accuracy, followed by the Multiclass Support Vector Machine with over 75% accuracy. Both of these models outperformed the Standard Convolution Neural Network that had nearly 74% accuracy of prediction.

3.2 The Classification Report

This is a machine learning tool that evaluates a model's performance by displaying key metrics for each class, including precision, recall, and F1-score. It provides a more detailed understanding of a model's accuracy based on all individual classes than a single overall score. The report is calculated by comparing the model's predictions against the true labels of the data.

3.2.1 SVM Classification Report

The report indicated good performance with the model in making predictions for the tomato dataset with over 80% f1-score but struggled with the cassava datasets with all metrics being lower than 70%.

SVM Classification Report				
	precision	recall	f1-score	support
Cassava_Mosaic	0.61	0.62	0.61	68
Tomato_Healthy	0.94	0.99	0.96	69
Tomato_Early_Blight	0.91	0.78	0.84	63
Cassava_Brown_Streak	0.66	0.62	0.63	65
Cassava_Healthy	0.62	0.65	0.63	62
Tomato_Bacterial_Spot	0.80	0.88	0.84	60
accuracy			0.75	387
macro avg	0.76	0.75	0.75	387
weighted avg	0.76	0.75	0.75	387

3.2.2 The Standard CNN Classification Report

The report indicated good performance with the model in making predictions for Cassava brown streak, cassava healthy and tomato bacterial spot with over 80% on all metrics but struggled with the other datasets with some metrics being lower than 60%.

CNN Classification Report				
	precision	recall	f1-score	support
Cassava_Mosaic	0.62	0.57	0.59	65
Tomato_Healthy	0.67	0.51	0.58	63
Tomato_Early_Blight	0.54	0.72	0.62	68
Cassava_Brown_Streak	0.96	0.88	0.92	60
Cassava_Healthy	0.94	0.75	0.83	63
Tomato_Bacterial_Spot	0.81	1.00	0.90	69
accuracy			0.74	388
macro avg	0.76	0.74	0.74	388
weighted avg	0.75	0.74	0.74	388

3.2.3 The ResNet-50 CNN Classification Report

The classification report indicated great performance with the model in making predictions for almost all classes of the datasets with over 70% f1-score.

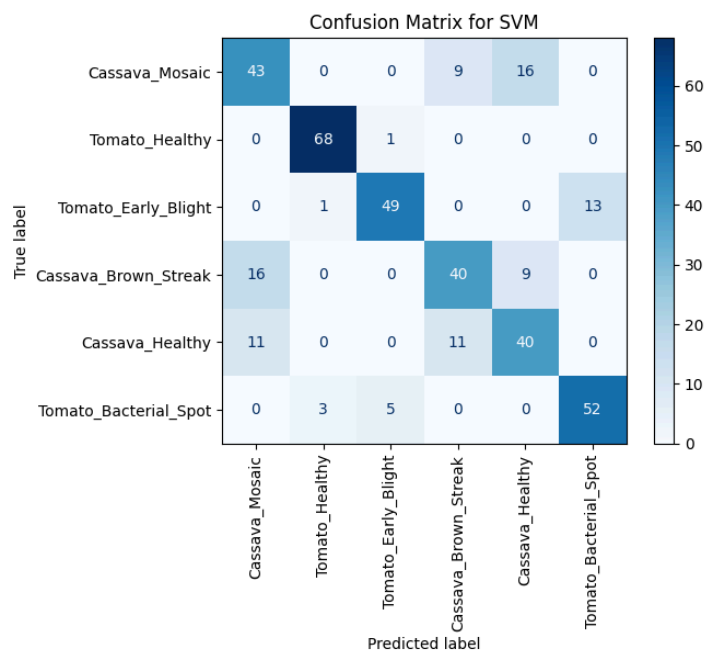
ResNet50 Classification Report				
	precision	recall	f1-score	support
Cassava_Mosaic	0.76	0.82	0.79	65
Tomato_Healthy	0.80	0.97	0.88	63
Tomato_Early_Blight	0.98	0.71	0.82	68
Cassava_Brown_Streak	0.98	0.93	0.96	60
Cassava_Healthy	0.64	1.00	0.78	63
Tomato_Bacterial_Spot	1.00	0.54	0.70	69
accuracy			0.82	388
macro avg	0.86	0.83	0.82	388
weighted avg	0.86	0.82	0.82	388

3.3 The Confusion Matrix

This is a table used to evaluate the performance of a classification model by comparing the actual class labels with the predicted class labels to show how many instances were correctly and incorrectly classified. It's useful for identifying where the model is confused, as the diagonal elements represent correct predictions, and off-diagonal elements represent errors.

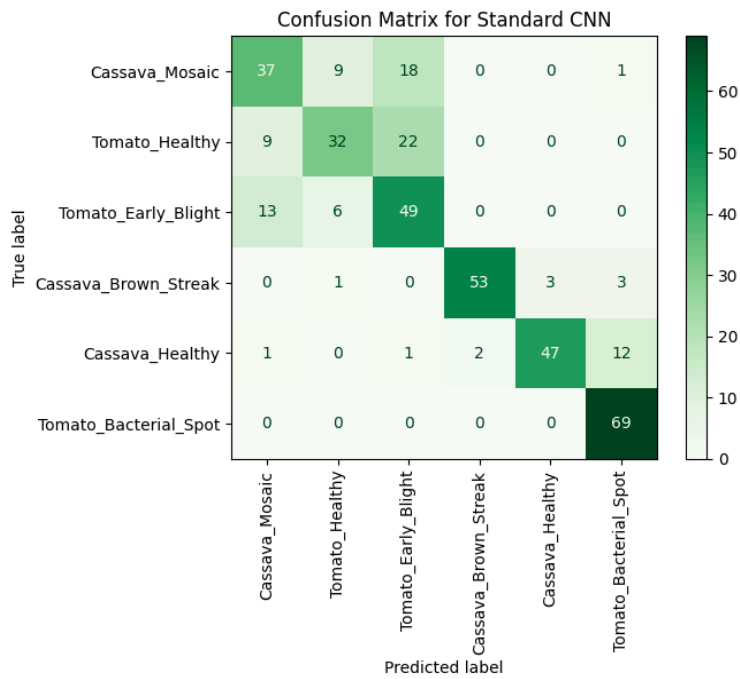
3.3.1 SVM Confusion Matrix

The confusion matrix indicated good performance with the model in making the right predictions for all the classes but struggled with predicting the right labels for the cassava dataset with nearly 16 misclassifications. It also struggled determining the true label of tomato early blight with 13 misclassifications as tomato bacterial spot disease.



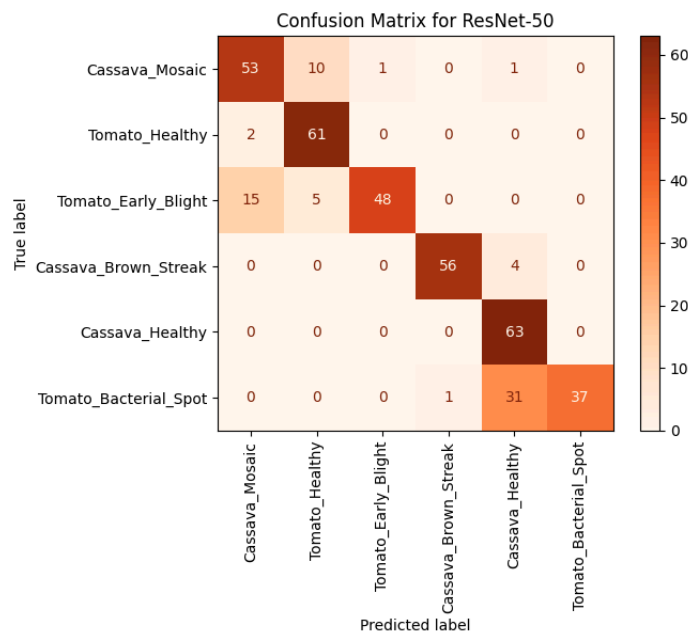
3.3.2 Standard CNN Confusion Matrix

The matrix indicated good performance with the model in making the right predictions for all the classes but struggled with predicting the right labels for cassava mosaic, healthy tomato, tomato early blight and healthy cassava.



3.3.3 ResNet50 Confusion Matrix

The matrix indicated great performance with the model in making the right predictions for all the classes and struggled with predicting the right labels for cassava mosaic and tomato early blight only.



3.4 Summary

Across all experiments, ResNet-50 achieved the strongest performance with an accuracy of nearly 82% and the highest F1-scores for most disease categories. The standard CNN achieved moderate performance, while the SVM despite a better accuracy score than the standard CNN performed least,

particularly on cassava disease classes. Confusion matrices showed that ResNet made fewer cross-class errors. These results demonstrate that transfer learning with deep CNN architectures is more effective for leaf disease detection than classical machine learning or shallow convolutional models.

4.0 Conclusion

This project set out to develop and evaluate machine learning models capable of detecting key diseases affecting cassava and tomato crops in Uganda. Using datasets from Makerere AI Lab, NaCRRI and PlantVillage, three supervised learning models: Multiclass Support Vector Machine (SVM), a Standard Convolutional Neural Network (CNN), and a pretrained ResNet-50 CNN were trained and compared across multiple performance metrics including accuracy, precision, recall, F1-score, and confusion matrices.

The results demonstrate that ResNet-50 achieved the highest performance, attaining over 81% accuracy, followed by the multiclass SVM and the standard CNN. ResNet-50 also produced consistently higher F1-scores across most disease classes and showed fewer misclassifications overall, indicating stronger generalization and robustness in identifying complex disease patterns.

These findings show that deep pretrained CNN architectures outperform traditional machine learning and shallow CNN models on multiclass leaf disease detection tasks especially when training resources, time, and dataset size are limited.

The project therefore demonstrates that machine learning when combined with accessible datasets can meaningfully support early disease detection in cassava and tomatoes, contributing to improved yields, reduced losses, and enhanced farmer livelihoods. With further development, the system could be integrated into a mobile or offline-first tool that enables real-time field diagnosis for smallholder farmers.

5.0 Future Work

Future work will focus on expanding the dataset to include more images taken under diverse field conditions, improving model robustness, and reducing misclassification among visually similar diseases. Additional enhancement will involve training more advanced architectures such as EfficientNet or Vision Transformers (ViT). Deploying the best-performing model in a lightweight mobile application using TensorFlow Lite or PyTorch Mobile to enable real-time diagnosis in rural areas with limited connectivity.